UNIVERSITÀ DI BOLOGNA



School of Engineering

Master Degree in Automation Engineering

Optimal Control and Reinforcement Learning

Optimal Control of a Gymnast Robot

Professor: **Giuseppe Notarstefano**

Students:
Alessandro Gaggioli
Mattia Gamberini
Andrea Gennari

Academic year 2025/2026

Abstract

This project addresses the optimal control problem of a planar Acrobot. The primary objective is to model the system dynamics of the double pendulum. The dynamics is modeled and discretized using the CasADi framework. The project is divided into three main phases: trajectory generation, trajectory tracking via LQR, trajectory tracking via MPC. First, an optimal trajectory is generated using a closed-loop Newton's method provided with an Armijo line-search strategy, using two approaches: a step reference and a 3-order smooth reference curve. Subsequently, trajectory tracking is implemented using a Linear Quadratic Regulator (LQR) based on time-varying linearization. Finally, a Model Predictive Control (MPC) scheme is designed to handle state and input constraints explicitly. Simulation results validate the effectiveness of the proposed methods.

Contents

Introduction **5**

1 Task 0 – Problem setup **7**

2 Task 1 – Trajectory generation **9**
2.1 Equilibria and Reference Curve 9
2.2 Optimal Trajectory 10
2.3 Solving procedure 10
2.4 Step-size selection rule 12
2.5 Armijo iterations Plots 13
2.6 Results 15

3 Task 2 - Smooth trajectory generation **21**
3.1 Smooth Reference Curve 21
3.2 Armijo iterations and Optimal Trajectory Plots 22
3.3 Results 29

4 Task 3 - Trajectory tracking via LQR **30**
4.1 Dynamic Programming 30
4.2 LQR Implementation 32
4.3 Results 32

5 Task 4 - Trajectory tracking via MPC **34**
5.1 Introduction to MPC 34
5.2 Implementation 34
5.3 Results 35
5.4 Better MPC constraint example 38

Conclusions **41**

Bibliography **42**

Introduction

In this project, we consider a planar gymnast robot, known as the Acrobot, modeled as a double pendulum with actuation provided only at the joint between the two links. The goal is to design and implement an optimal control law using Newton's method to perform a swing maneuver between two equilibria. The tracking of the obtained optimal trajectory is performed using LQR and MPC methods.

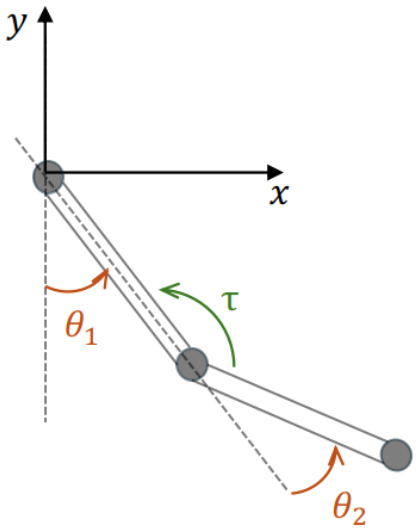


Figure 1: Model of an Acrobot

Contributions

- **Dynamics Discretization:**The non linear dynamics of the double pendulum have been modeled and discretized using **CasADi** framework.
- **Optimal Trajectory Generation:** A Newton-based optimal control algorithm to generate feasible trajectories. A equilibrium-to-equilibrium transition and a smooth reference curve.
- **Trajectory Tracking via LQR:** A Linear Quadratic Regulator designed based on the time-varying linearization of the system along the optimal trajectory.
- **Trajectory Tracking via MPC:** A Model Predictive Control scheme implemented to track the reference trajectory, handling the prediction horizon and re-optimization at each time step.
- **Animation:** An animation is produced executing the trajectory tracking via LQR.

Chapter 1

Task 0 – Problem setup

Taking into account the dynamics of the Acrobot, the state space consists of $x = (\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)^\top$, where θ_1 is the angle of the first joint w.r.t. the vertical reference and θ_2 is the relative angle of the second link. The control input is defined as $u(t) = \tau$, which represents the torque applied to the second link. The continuous dynamic model is given by the following equation:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + F\dot{q} + G(q) = \begin{pmatrix} 0 \\ \tau \end{pmatrix}$$

The matrices are defined as follows:

$$M = \begin{pmatrix} I_1 + I_2 + l_{c1}^2 m_1 + m_2(l_1^2 + 2l_1 l_{c2}\cos(\theta_2) + l_{c2}^2) & I_2 + l_{c2} m_2(l_1 \cos(\theta_2) + l_{c2}) \\ I_2 + l_{c2} m_2(l_1 \cos(\theta_2) + l_{c2}) & I_2 + l_{c2}^2 m_2 \end{pmatrix}$$

$$C = \begin{pmatrix} -l_1 l_{c2} m_2 \dot{\theta}_2 \sin(\theta_2) & -l_1 l_{c2} m_2 (\dot{\theta}_1 + \dot{\theta}_2) \sin(\theta_2) \\ l_1 l_{c2} m_2 \dot{\theta}_1 \sin(\theta_2) & 0 \end{pmatrix}$$

$$G = \begin{pmatrix} g l_{c1} m_1 \sin(\theta_1) + g m_2 (l_1 \sin(\theta_1) + l_{c2} \sin(\theta_1 + \theta_2)) \\ g m_2 l_{c2} \sin(\theta_1 + \theta_2) \end{pmatrix}$$

Here, l_{ci} denotes the distance to the center of mass, m_i the mass, l_i the length, and I_i the moment of inertia for the i-th link. The dynamics of the Acrobot are continuous and non-linear, requiring discretization to formulate the optimization problem.
Although the problem setup initially suggested using a standard discretization scheme like Runge-Kutta 4th order (RK4), initial tests showed that an explicit Python implementation was computationally expensive and slow for the iterative optimization process.
Consequently a powerful framework for algorithmic differentiation and numerical optimization, CasADi were adopted. Specifically, the continuous

dynamics $\dot{x} = f_c(x, u)$ are implemented symbolically using CasADi's expression graph. Instead of a manual fixed-step integrator, we utilized the CVODES integrator included in CasADi. This allows for high-precision integration of the differential-algebraic equations over the discretization interval dt. CasADi allows for the exact computation of the discrete-time Jacobians

$$A_t = \frac{\partial x_{t+1}}{\partial x_t} \qquad\qquad B_t = \frac{\partial x_{t+1}}{\partial u_t}$$

via algorithmic differentiation. This approach is significantly faster and more accurate than finite differences or symbolic lambdification via SymPy, permitting a drastic reduction of the computational time.

To address this, we implemented the dynamics using the **CasADi** framework. The state transition is defined as:

$$x_{t+1} = \int_t^{t+dt} f_c(x(\tau), u_t)\, d\tau$$

The problem consists to moving from one equilibrium to another. For the stage and terminal costs, a quadratic form is used:

$$l_t = \frac{1}{2}\left((x_t - x_t^{ref})^\top Q_t (x_t - x_t^{ref}) + (u_t - u_t^{ref})^\top R_t (u_t - u_t^{ref})\right)$$
$$l_T = \frac{1}{2}\left((x_T - x_T^{ref})^\top Q_T (x_T - x_T^{ref})\right)$$

Chapter 2

Task 1 – Trajectory generation

2.1 Equilibria and Reference Curve

Compute two equilibria for your system and define a reference curve between the two.

Two equilibria configurations are computed exploiting least square method applied to the dynamics. The problem has two equation with three unknowns, so the result are the best uncontrolled angle θ_1 and input torque τ, able to minimize the imposed controlled angle θ_2. Numerical values are shown in the following system's equations. The position reference curve consists of a time-symmetrical step curve between the two equilibria.

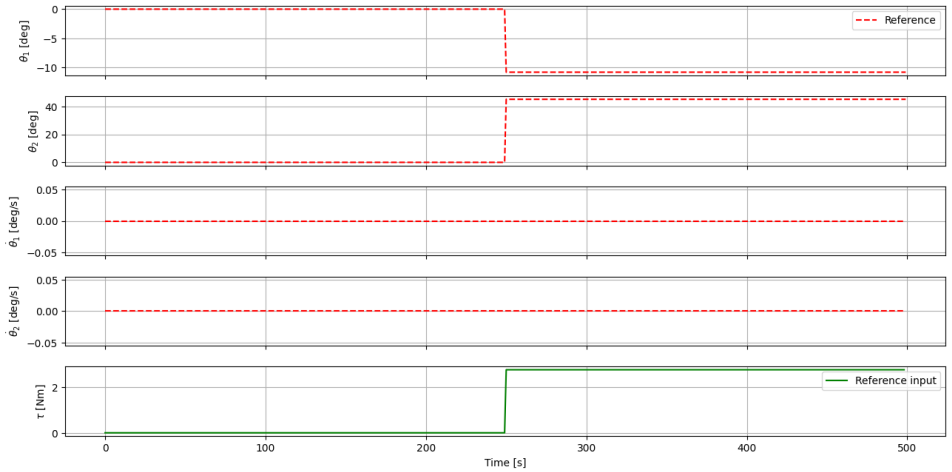


Figure 2.1: Reference states (θ_1, θ_2, $\dot{\theta}_1$, $\dot{\theta}_2$) and input τ

$$\theta_{1,\text{eq}} = 0, \qquad \theta_{2,\text{eq}} = 0, \qquad u_{\text{eq}} = 0$$
$$\theta_{1,\text{eq}} = -10.800, \qquad \theta_{2,\text{eq}} = 45, \qquad u_{\text{eq}} = 2.757$$

2.2 Optimal Trajectory

Compute the optimal transition to move from one equilibrium to another exploiting the Newton's like algorithm (in closed-loop version) for optimal control.

In this task, the objective is to compute an optimal transition trajectory for the gymnast robot (Acrobot) to move from one equilibrium configuration to another. To solve this nonlinear optimization problem, a closed-loop version of the Newton's Method for Optimal Control has been implemented.

Newton's algorithm in closed loop configuration is used to minimize a cost that penalizes the deviation from the desired trajectory and excessive control effort.

The finite-horizon optimal control problem consists on minimizing the following cost function, composed of a stage cost $\ell_t(\cdot)$ term and a terminal cost $\ell_T(\cdot)$ term:

$$J(\mathbf{x}, \mathbf{u}) = \sum_{t=0}^{T-1} \ell_t(\mathbf{x}_t, \mathbf{u}_t) + \ell_T(\mathbf{x}_T)$$

These terms are quadratic forms weighted by the matrices Q, R, and Q_T, respectively, which are design parameters of the controller that must be tuned for the specific system, to reach the required system's performance.

Greater emphasis could be placed on reducing position and velocity errors if tighter trajectory tracking is required. On the other side, limiting the control input by increasing input cost matrix can be useful in a limited stored energy system such as a battery powered one.

As per project specifications, a *Gauss-Newton* approximation was used, neglecting the second-order terms related to the dynamics in the update of P_{t+1} and λ_{t+1}.

2.3 Solving procedure

The optimal control problem is solved exploiting Newton's method in closed loop, by following the two steps procedure explained below.

Step 1) Compute descent direction
This step is based on the solving procedure of an affine LQR problem, that

starts with linearization of the dynamics around the reference trajectory, by computing system and cost gradients.

$$\nabla_x f_t(x_t^k, u_t^k), \nabla_u f_t(x_t^k, u_t^k), \nabla_x \ell_t(x_t^k, u_t^k), \nabla_u \ell_t(x_t^k, u_t^k), \nabla \ell_T(x_T^k) \qquad (2.1)$$

The costate vector is recursively evaluated starting from $t = T$:

$$\begin{aligned} \lambda_T &= \nabla \ell_T(x_T) \\ \lambda_t &= \mathbf{q}_t + A_t^\top \lambda_{t+1} \end{aligned}$$

The equation for λ_t corresponds to the standard costate equation used to compute the gradient of the cost function along the trajectory.

Difference Riccati Equation (DRE) is solved backward in time from instant T to 0 and feedback gain matrix K_t and feedforward terms σ_t are obtained:

$$\begin{aligned} K_t &= -\left(R_t + B_t^\top P_{t+1} B_t\right)^{-1} \left(S_t + B_t^\top P_{t+1} A_t\right), \\ \sigma_t &= -\left(R_t + B_t^\top P_{t+1} B_t\right)^{-1} \left(\mathbf{r}_t + B_t^\top p_{t+1} + B_t^\top P_{t+1} c_t\right), \\ p_t &= q_t + A_t^\top p_{t+1} + A_t^\top P_{t+1} c_t - K_t^\top \left(R_t + B_t^\top P_{t+1} B_t\right) \sigma_t, \\ P_t &= Q_t + A_t^\top P_{t+1} A_t - K_t^\top \left(R_t + B_t^\top P_{t+1} B_t\right) K_t, \end{aligned}$$

with p_T=q_T and $P_T = Q_T$.

Starting from the optimal solution, a rollout is performed on the linearized dynamics to propagate the system states.

$$\begin{aligned} \Delta u_t^* &= K_t^* \Delta x_t^* + \sigma_t^* \qquad\qquad t = 0, \ldots, T-1, \\ \Delta x_{t+1}^* &= A_t \Delta x_t^* + B_t \Delta u_t^* \end{aligned}$$

The descent direction for the $k - th$ iteration is evaluated as follows:

$$\text{descent_arm} \;=\; \nabla J(u^k)^\top \Delta u^k$$

It is relevant to specify that K_t and σ_t gains have been evaluated onto the trajectory-linearized system and then applied to the non linear original system in the next step.

Step 2) Compute the new state-input trajectory
It exploits the closed loop Newton's method for optimal control, which basically evaluate the new state trajectory by applying the computed controllers gains K_t and σ_t onto the real non linear dynamics, by forward integration with the new control law:

$$u_t^{k+1} = u_t^k + \gamma^k(\sigma_t^k + K_t^k \Delta x_t^k) + K_t^k(x_t^{k+1} - x_t^k - \gamma^k \Delta x_t) \quad t = 0, \ldots, T-1,$$
$$x_{t+1}^{k+1} = f_t(x_t^{k+1}, u_t^{k+1})$$

To ensure convergence and prevent overly large updates that may destabilize the trajectory, a step-size selection rule is used, such as the Armijo rule.

The algorithm ends when the improvement in the total cost function between two successive iterations falls below a predefined threshold or reaches the maximum iteration limit.

2.4 Step-size selection rule

The selected step size selection algorithm exploits Armijo rule, the which iterative application seeks for cost function reduction and ensure that each new trajectory results in a decrease of the overall cost. The algorithm searches for the maximum γ, such that the Armijo condition is respected:

$$J(\mathbf{u}^{new}) \leq J(\mathbf{u}^k) + c \cdot \gamma \cdot \nabla J \Delta \mathbf{u}$$

where c is a tolerance parameter (set to 0.5).

The plots of the Armijo line search are shown below and allow to verify the correctness of the gradient computation and the effectiveness of the step-size selection. The green line represents the actual nonlinear cost $J(\mathbf{u} + \gamma \mathbf{d})$ as a function of the step-size γ. The red line shows the expected cost reduction based on the local linearization ($\nabla J^\top \mathbf{d}$). The blue stars indicate the trial steps performed by the backtracking line search algorithm.

2.5 Armijo iterations Plots

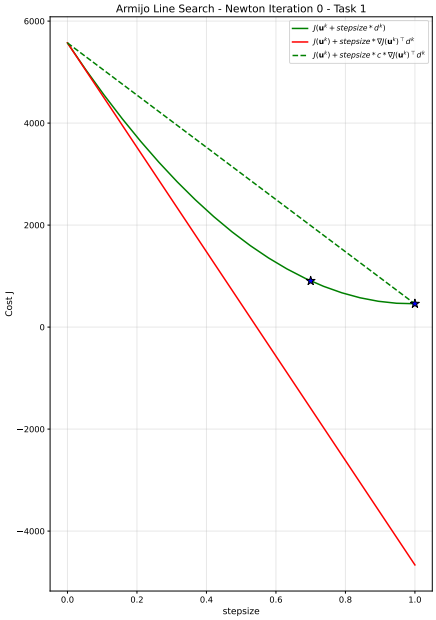


Figure 2.2: Armijo Search Iteration 0

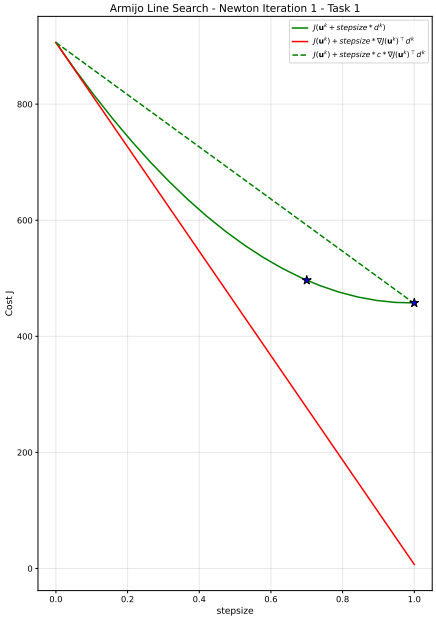


Figure 2.3: Armijo Search Iteration 1

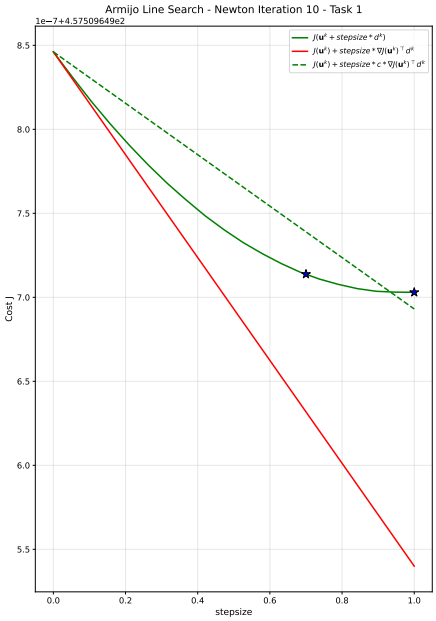


Figure 2.4: Armijo Search Iteration 10

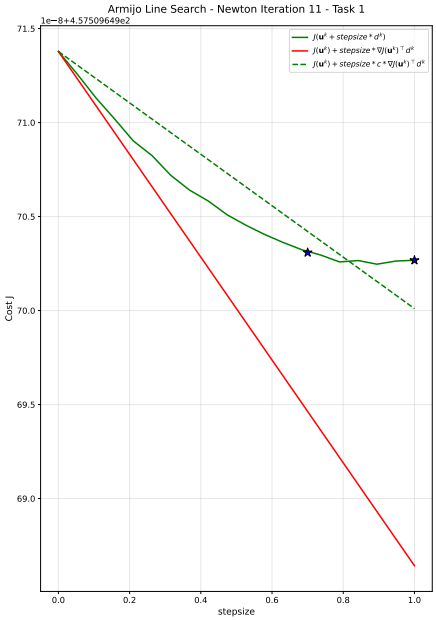


Figure 2.5: Armijo Search Iteration 11

2.6 Results

As shown in the following plots, the simulated system is unable to track the step reference due to the system inertia. Nevertheless, a clear improvement in the trajectories can be observed as the Newton's algorithm iterations progress, thanks to lower tracking error and decreasing cost.
As the trajectory evolves a significant disparity emerges between θ_1 and θ_2, even if the controller reacts trying to steer positions and velocities toward the desired trajectory. This behavior is a consequence of the system's underactuated nature, but thanks to optimal control the actuated joint try to make up for the non-actuated one, also depending on the imposed weights.

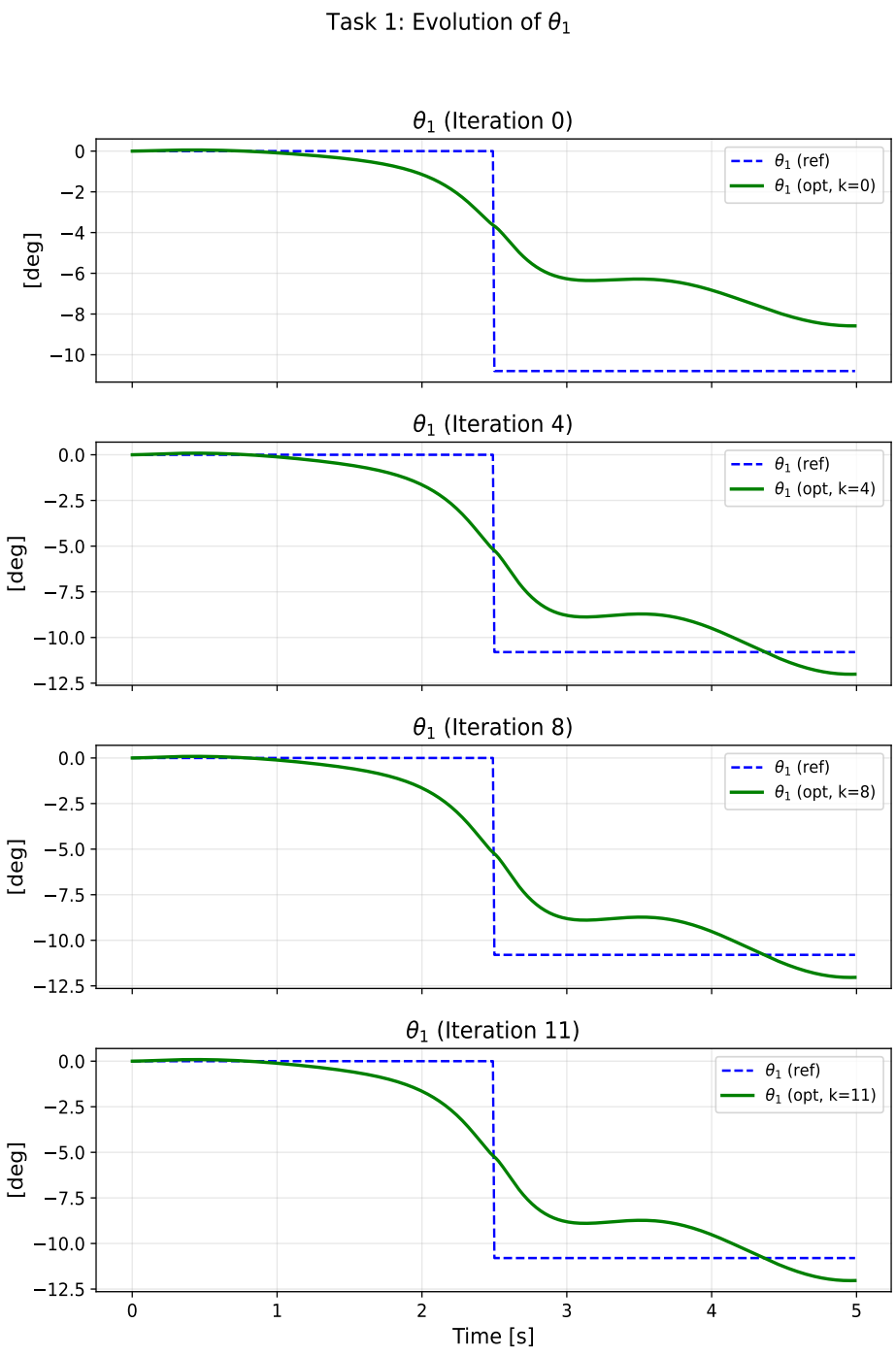


Figure 2.6: Evolution of θ_1.

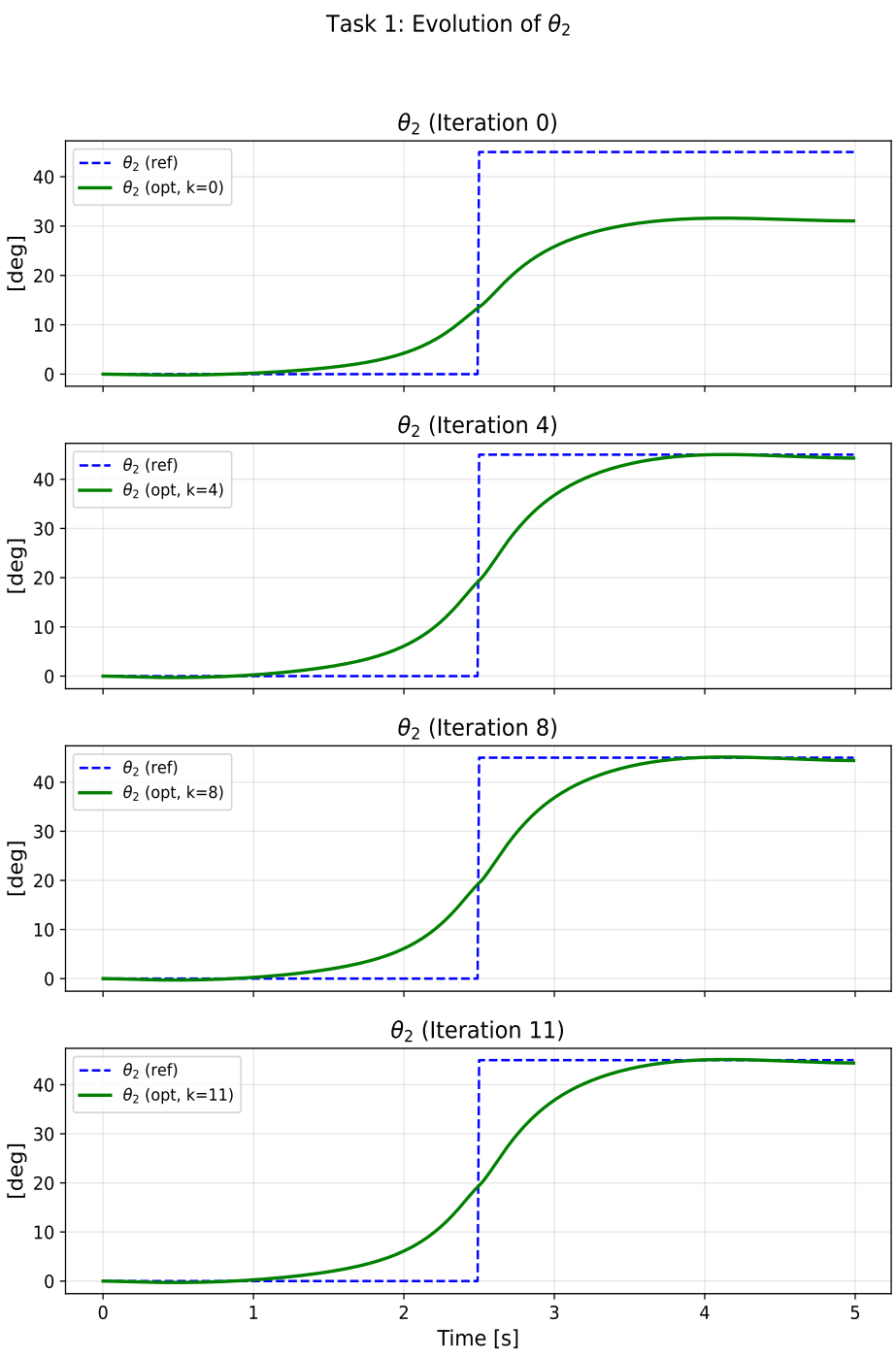


Figure 2.7: Evolution of θ_2.

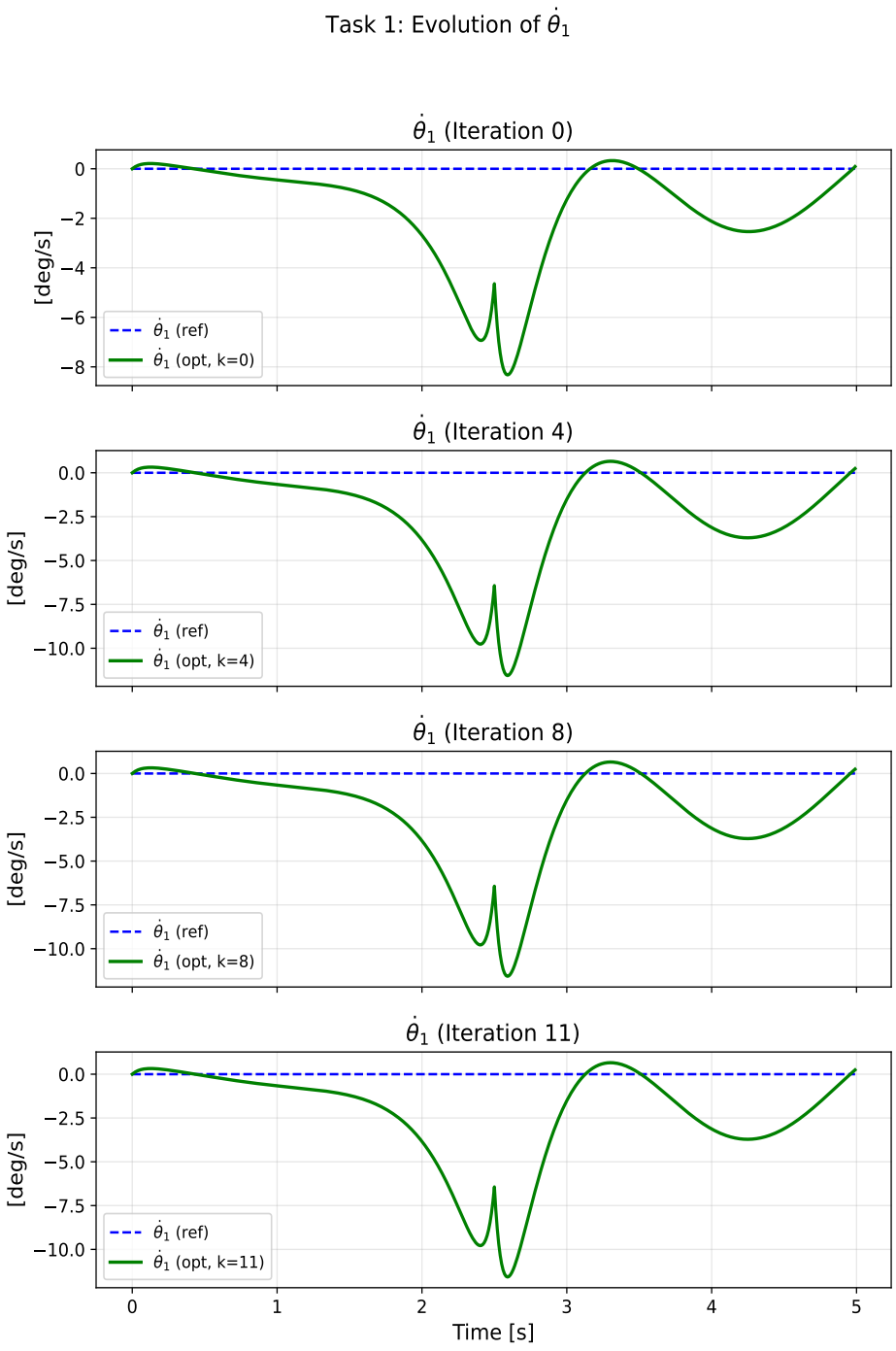


Figure 2.8: Evolution of $\dot{\theta}_1$.

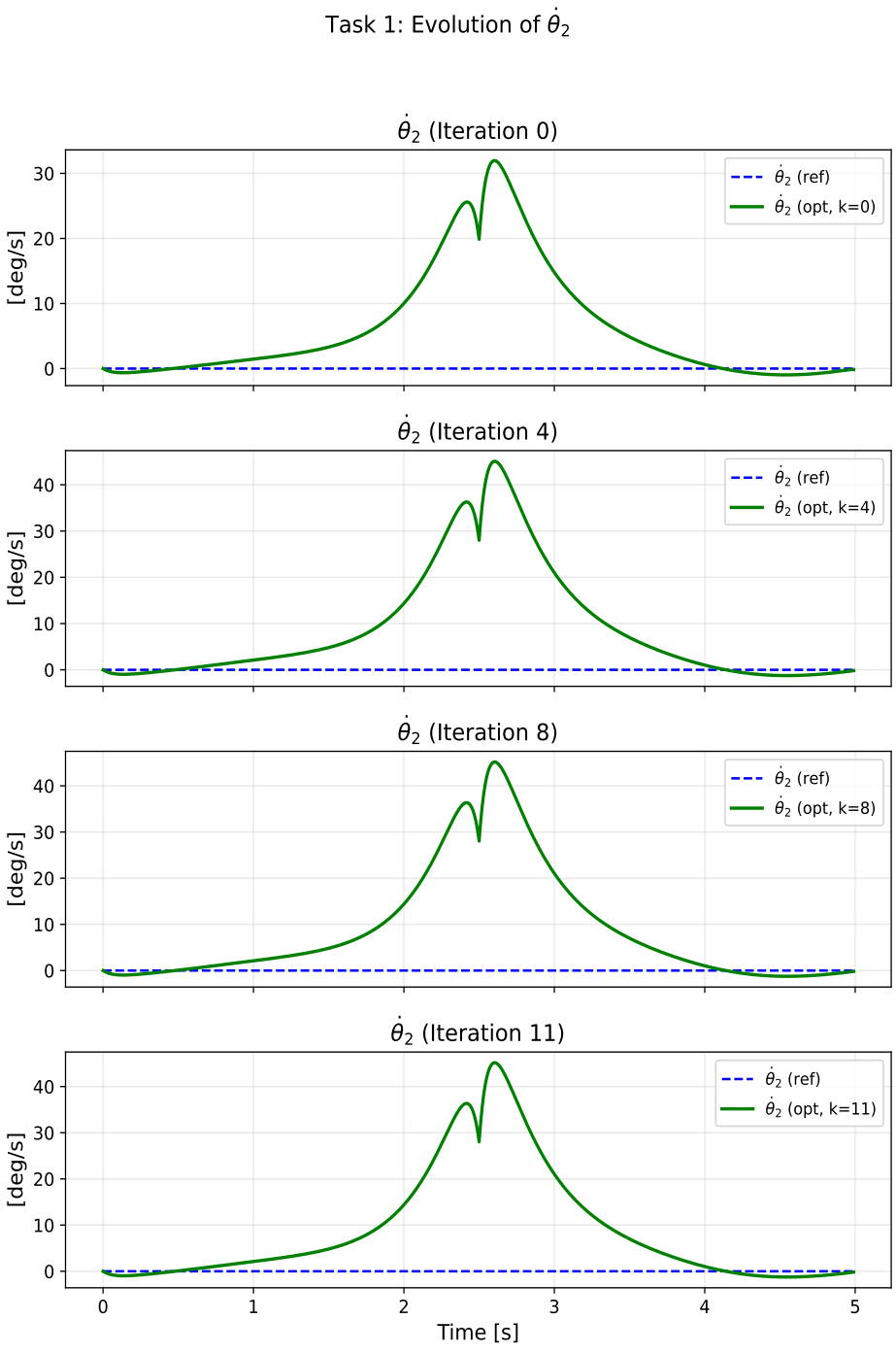


Figure 2.9: Evolution of $\dot{\theta}_2$.

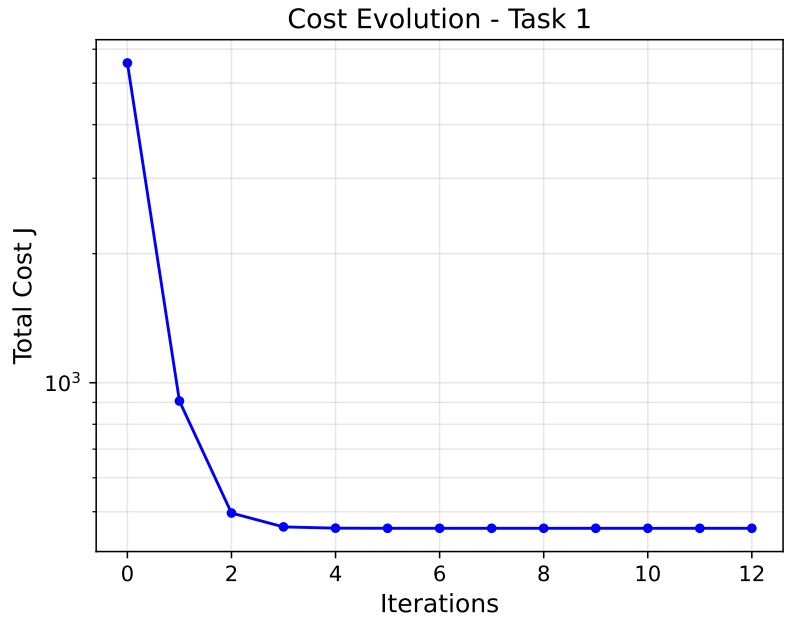


Figure 2.10: Cost Evolution

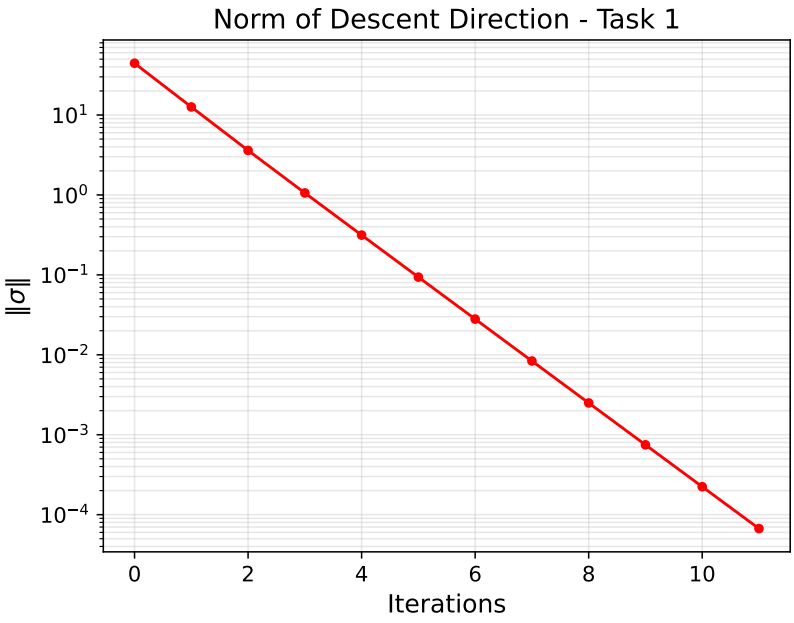


Figure 2.11: Descent Norm

Chapter 3

Task 2 - Smooth trajectory generation

3.1 Smooth Reference Curve

Generate a desired smooth state-input curve and perform the trajectory generation task on this new desired curve.

In this task, the reference trajectory is composed of two constant reference segments corresponding to equilibrium points identified in Task 1, connected by a third-order polynomial that ensures smooth transitions between them.
This reference trajectory is commonly referred to as a **quasi-static trajectory**, as it represents a slow and continuous evolution of the system states in which the system remains at, or very close to, equilibrium at all times.
Under quasi-static conditions, dynamic effects such as transients and inertial forces are negligible, since the system has sufficient time to adapt to the changing reference while maintaining balance.

In Task 2, the same optimal control problem is addressed as in Task 1, with the distinction that the reference curve from which the system starts is a smooth transition between equilibrium points.
The chosen cost matrices are reported below and are designed to place greater emphasis on the under-actuated variable, indirectly regulating it through the actuated one.

$$Q_t = \begin{pmatrix} 100 & 0 & 0 & 0 \\ 0 & 50 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 10 \end{pmatrix}, \quad R_t = \begin{pmatrix} 1 \end{pmatrix}, \quad Q_T = \begin{pmatrix} 2000 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 \\ 0 & 0 & 200 & 0 \\ 0 & 0 & 0 & 200 \end{pmatrix}$$

3.2 Armijo iterations and Optimal Trajectory Plots

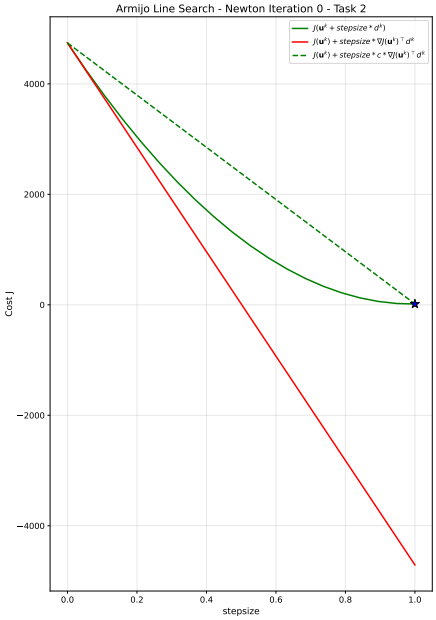


Figure 3.1: Armijo Search Iteration 0

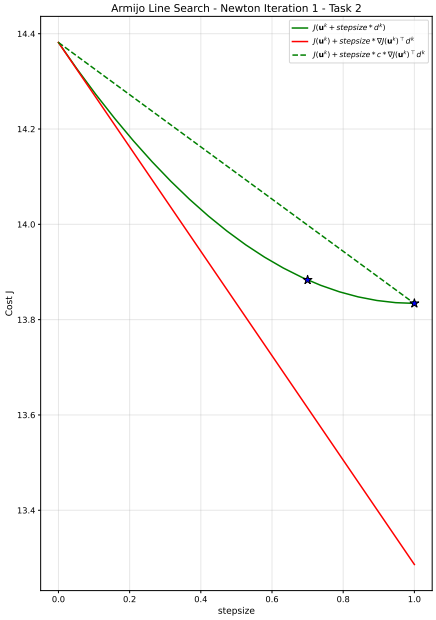


Figure 3.2: Armijo Search Iteration 1

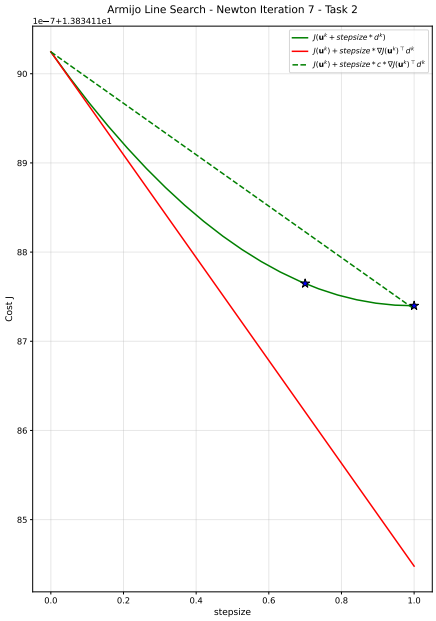


Figure 3.3: Armijo Search Iteration 7

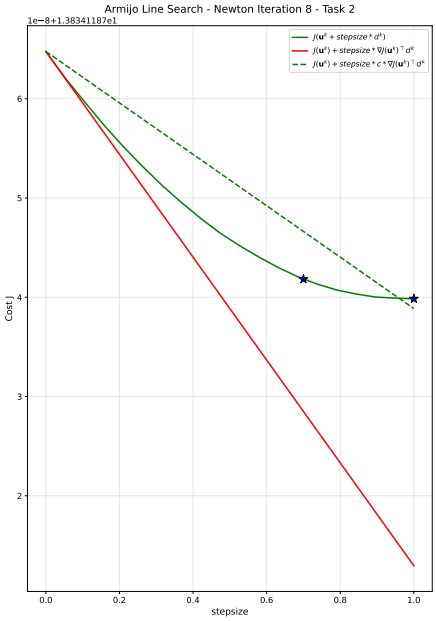


Figure 3.4: Armijo Search Iteration 8

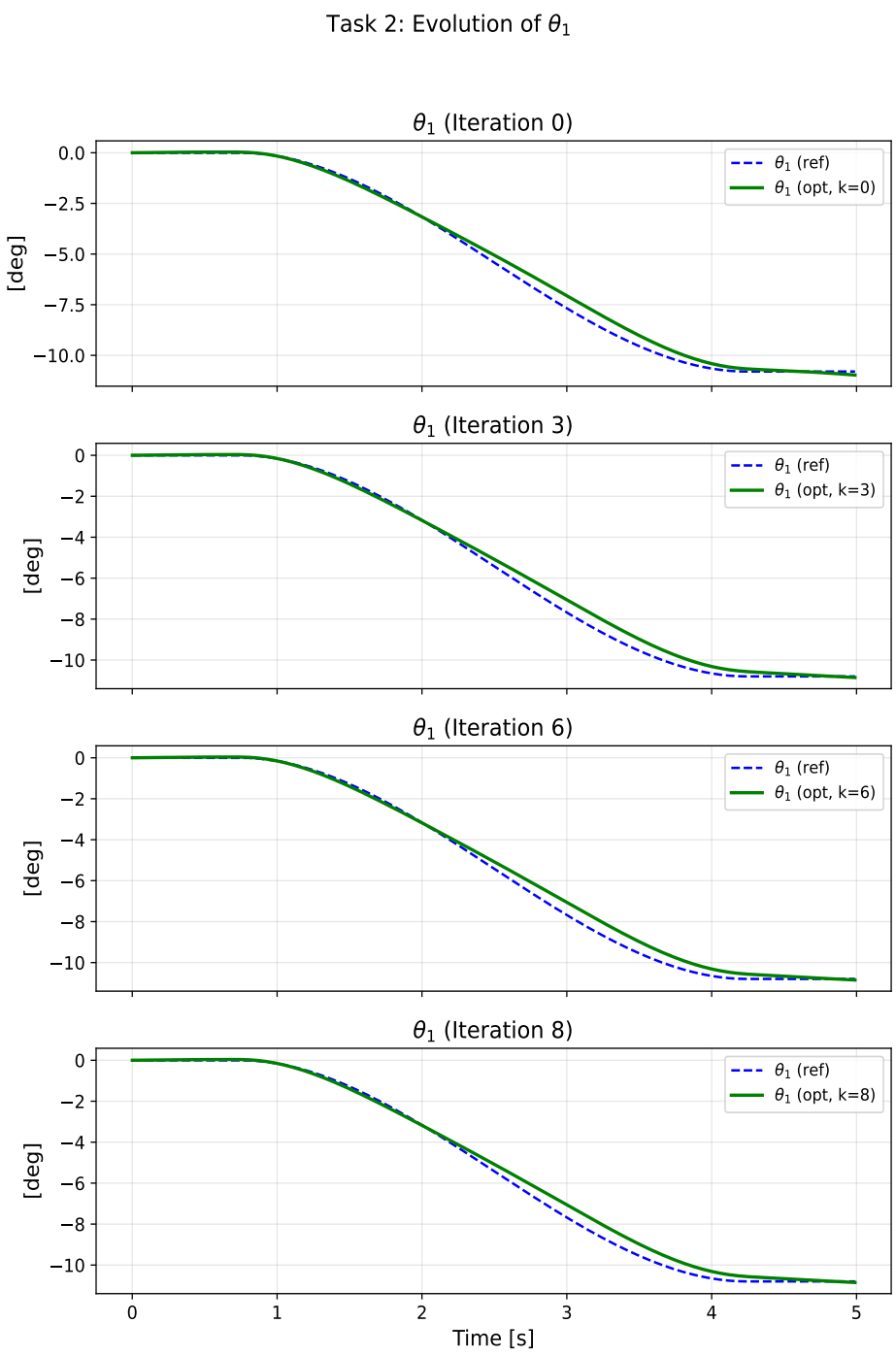


Figure 3.5: Evolution of θ_1.

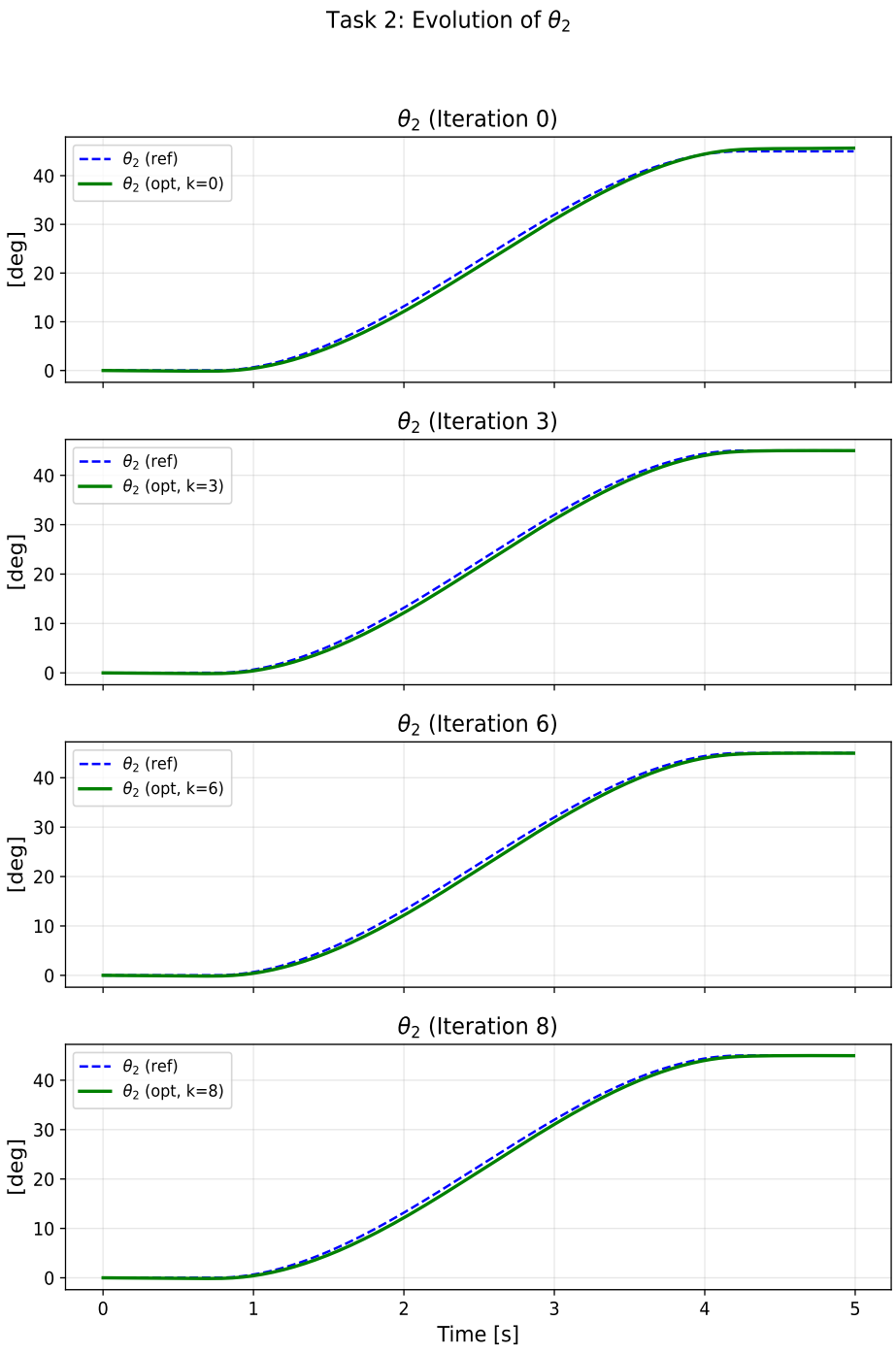


Figure 3.6: Evolution of θ_2.

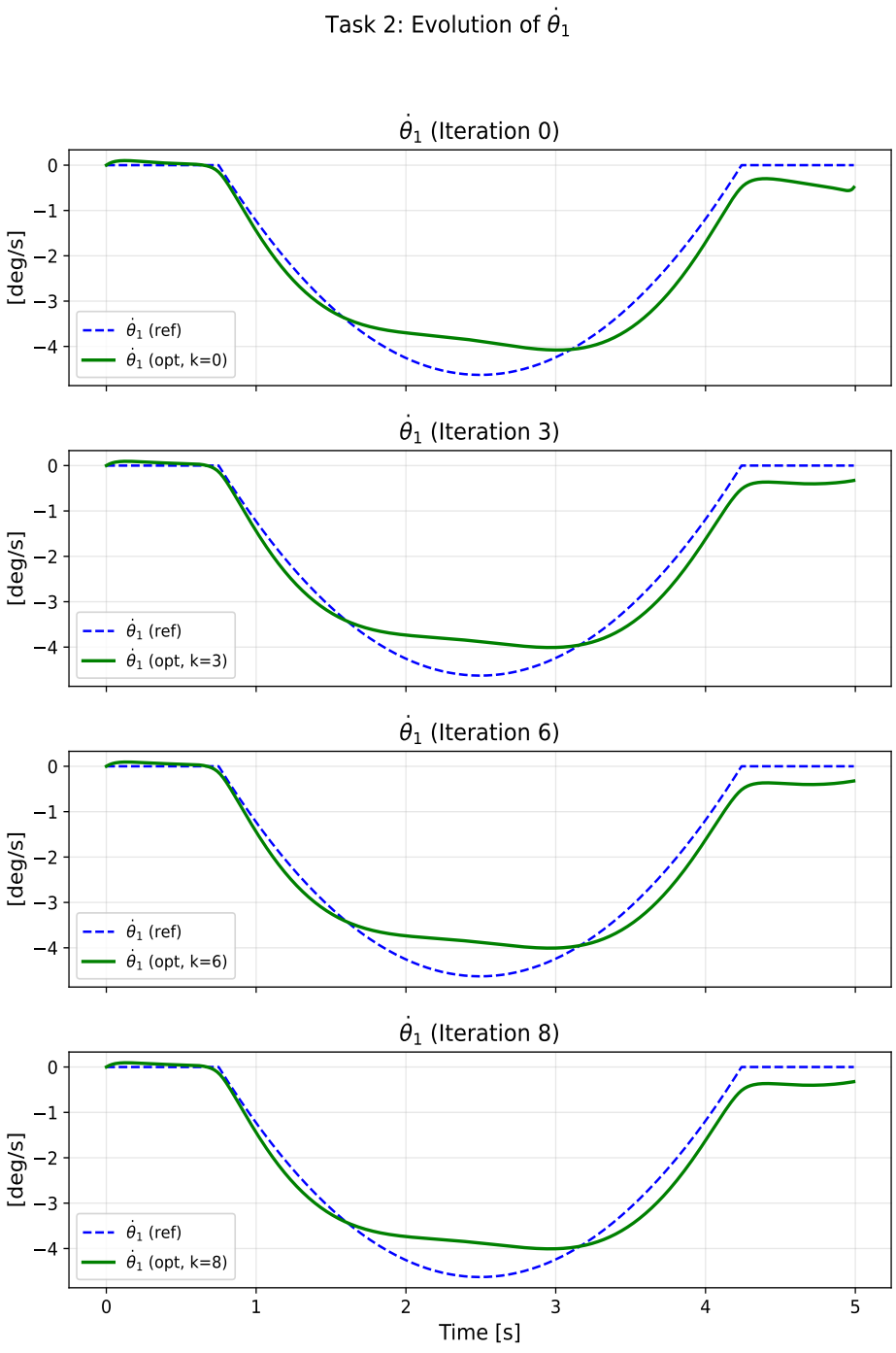


Figure 3.7: Evolution of $\dot{\theta}_1$.

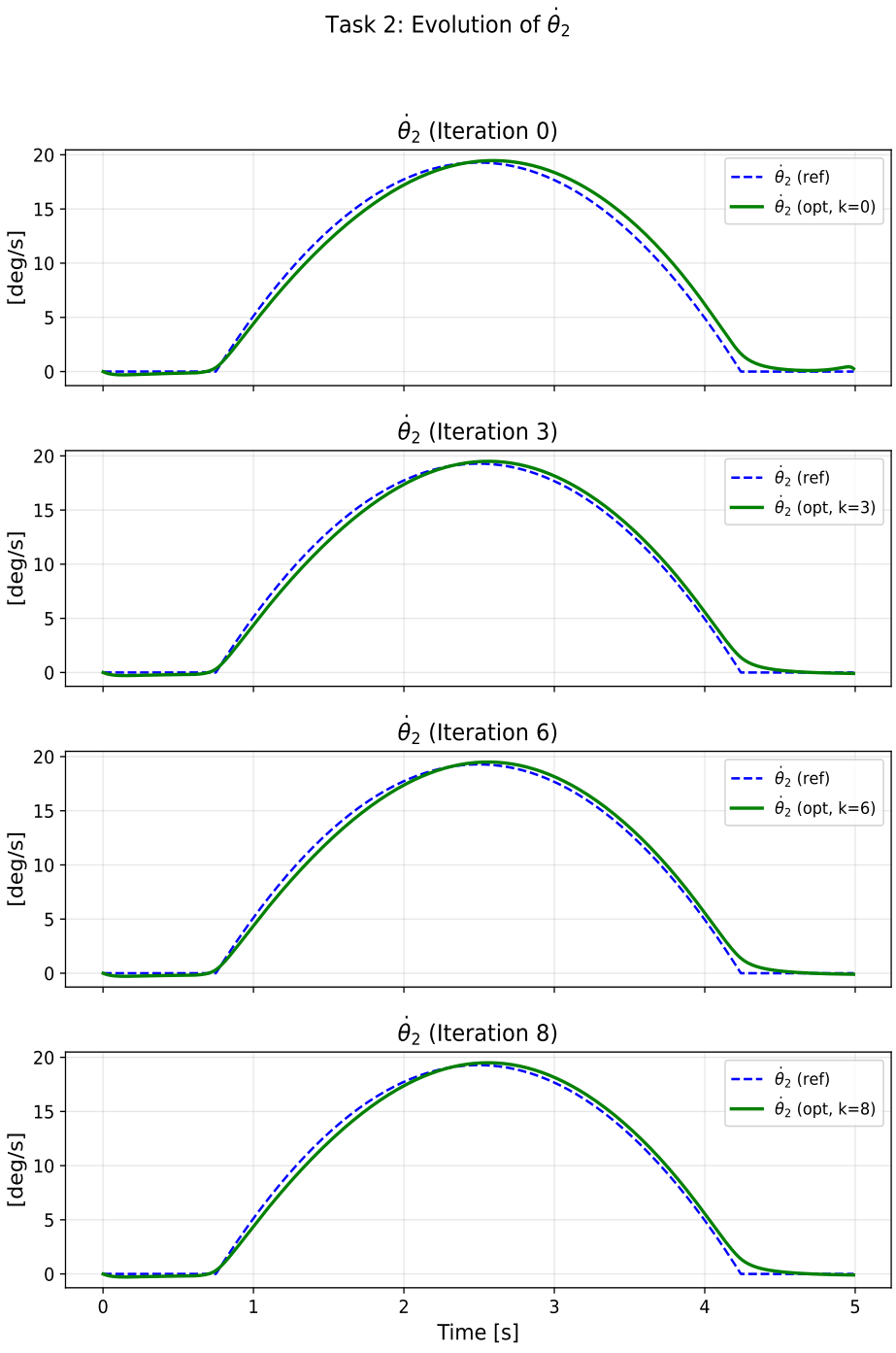


Figure 3.8: Evolution of $\dot{\theta}_2$.

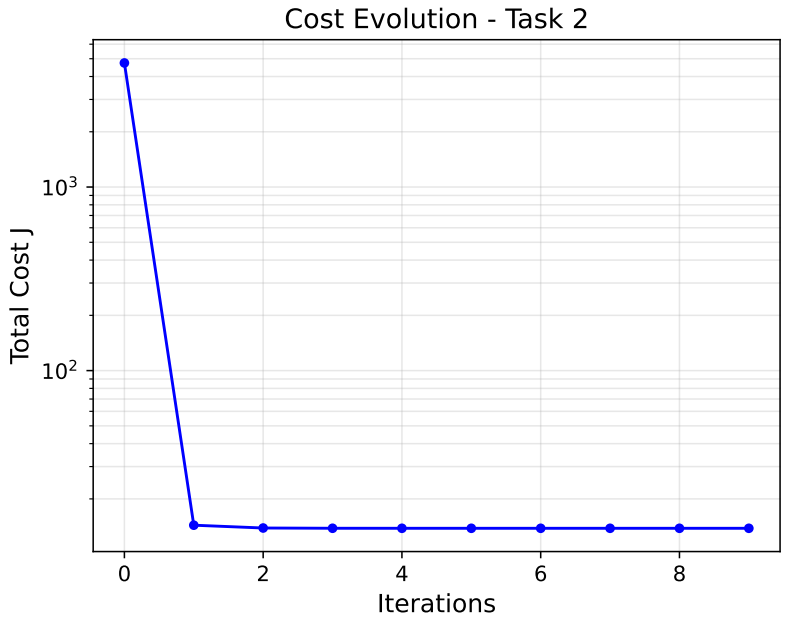


Figure 3.9: Cost Evolution

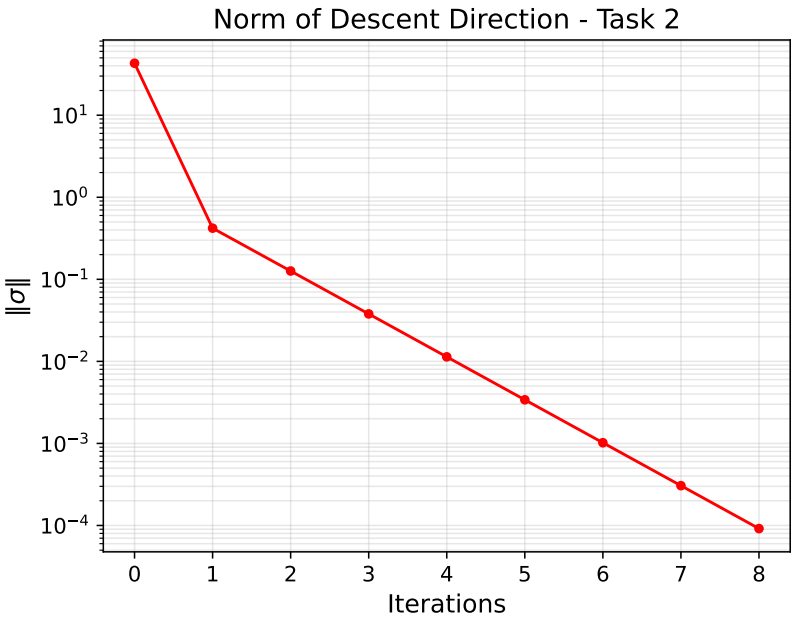


Figure 3.10: Descent Norm

3.3 Results

The smoothed reference curve permits better tracking performance and smoother actuation, preventing input peaks or discontinuous velocity profiles.

This is the demonstration that in optimal control problems, quasi-static trajectories can simplify the system dynamics and prevent abrupt state variations. As a result, closed-loop stability is improved and the convergence of numerical optimization algorithms is significantly enhanced.

Chapter 4

Task 3 - Trajectory tracking via LQR

Linearizing the robot dynamics about the generated trajectory (x^{gen}, u^{gen}) computed in Task 2, exploit the LQR algorithm to define the optimal feedback controller to track this reference trajectory.

In particular, you need to solve the LQ Problem:

$$\min_{\substack{\Delta x_1,\dots,\Delta x_T \\ \Delta u_0,\dots,\Delta u_{T-1}}} \sum_{t=0}^{T-1} \left(\Delta x_t^\top Q^{\text{reg}} \Delta x_t + \Delta u_t^\top R^{\text{reg}} \Delta u_t \right) + \Delta x_T^\top Q_T^{\text{reg}} \Delta x_T$$

$$\text{s.t. } \Delta x_{t+1} = A_t^{\text{gen}} \Delta x_t + B_t^{\text{gen}} \Delta u_t \qquad t = 0, \dots, T-1$$
$$x_0 = 0.$$

where A_t^{gen}, B_t^{gen} represents the linearization of the (nonlinear) system about the optimal trajectory. The cost matrices of the regulator are a degree-of-freedom of the designer under certain constraints.
Hint: to showcase the tracking performances, consider a perturbed initial condition, i.e., different than x_0.

4.1 Dynamic Programming

Dynamic Programming (DP) is an optimal control framework based on the concept of the optimal value function $V(x)$. Instead of performing a direct offline optimization over an entire trajectory, the problem is decomposed over time, by decomposing the global control problem defined over the horizon $[0, T]$ into a sequence of smaller subproblems defined on a moving time window of predefined horizon $[t, \tau]$. This decomposition is based on the Bellman's Principle of optimality, which states that any tail of an optimal trajectory is optimal as well for the corresponding subproblem characterized by same dynamics and terminal cost.

The primary limitation is the exponential growth in computational complexity as the state dimension increases, since the value function must be evaluated across the entire state space.

As great benefit, DP does not require differentiability or convexity assumption on cost and dynamic functions since it finds the global optimum by solving the Bellman Equation directly through value comparison, rather than relying on gradients or derivatives to determine the descent direction.

Mathematically, this principle is expressed through the Bellman equation:

$$V_t^*(\bar{x}) = \min_{u \in \mathbb{R}^m} \left[\ell_t(\bar{x}, u) + V_{t+1}^*\big(f_t(\bar{x}, u)\big) \right]$$

It is composed of an immediate (stage) cost and the optimal cost-to-go associated with the next state and it is evaluated backward in time. This formulation highlights that the optimal control law has a state-feedback structure.

The output of the dynamic programming algorithm is the optimal policy $\pi_t^*(x_t)$, which maps the current state to the control input minimizing the total cost:

$$\pi_t^*(x_t) = \arg\min_{u} \left[\ell_t(x_t, u) + V_{t+1}^*\big(f_t(x_t, u)\big) \right]$$

The algorithm can be summarized as follows:

1. Initialize the value function equal to the terminal cost and run **back propagation** of the Bellman's Equation:

$$\begin{aligned} V_T^*(\overline{x}) &= l_T(\overline{x}) \\ V_t^*(\bar{x}) &= \min_{u \in \mathbb{R}^m} \left[\ell_t(\bar{x}, u) + V_{t+1}^*\big(f_t(\bar{x}, u)\big) \right], \qquad t = T-1, \ldots, 0 \end{aligned}$$

2. Compute the policy:

$$\pi_t^*(x_t) = u_t^*$$

3. Compute the optimal state trajectory by **forward propagation** of the system dynamics:

$$x_{t+1}^* = f_t(x_t^*, u_t^*), \qquad t = 0, \ldots, T-1$$

4.2 LQR Implementation

The Linear Quadratic Regulator is a special case of the dynamic programming framework that requires linear dynamics systems and quadratic cost functions, namely LQ problems (Linear Quadratic). The resulting optimal controller has a linear state-feedback structure that could guarantee good stability properties. Note that LQR is evaluated onto error coordinates and it's result is a correction term of the control input

The solving procedure of the finite-horizon LTV-LQR problem inside the LQR.py file apply the following procedure.

1. Initialize the terminal condition $P_T = Q_T$
2. **Backward procedure** $t = T-1, \dots, 0$
   Linearize the system around the optimal trajectory, which was previously computed in task 2 and then stored in a dedicated folder *data*. Apply backward propagation of Difference Riccati Equation (DRE) to evaluate the cost-to-go matrices for all time steps:
   $$P_t = Q_t + A_t^\top P_{t+1} A_t - \big(S_t^\top + A_t^\top P_{t+1} B_t\big)\big(R_t + B_t^\top P_{t+1} B_t\big)^{-1}\big(S_t + B_t^\top P_{t+1} A_t\big)$$
   Evaluate of the optimal feedback gain as:
   $$K_t^\star := -\big(R_t + B_t^\top P_{t+1} B_t\big)^{-1}\big(S_t + B_t^\top P_{t+1} A_t\big)$$
3. **Forward procedure** $t = 0, \dots, T-1$
   Evaluate the feedback controller using the controller previously designed and then apply it to the original non linear dynamics:
   $$u_t = u_t^{traj} + K_t^{reg}(x_t - x_t^{traj})$$
   $$x_{t+1}^\star = A_t x_t^\star + B_t u_t^\star$$
   given $x_0^\star = x_{\text{init}}$

To evaluate the tracking performance, the closed-loop system is tested using a perturbed initial conditions ($x_0 = x_0^{ref} + \delta$) with $\delta = [1, 1]^\top$ degrees. The feedback controller should compensate for this deviation and drive the system toward the reference trajectory, minimizing the tracking error over time.

4.3 Results

As the trajectory evolves a significant disparity emerges between θ_1 and θ_2, even if the controller reacts trying to steer positions and velocities toward the desired trajectory. This behavior is a consequence of the system's underactuated nature, but thanks to optimal control the actuated joint try to make up for the non-actuated one, also depending on the imposed weights.

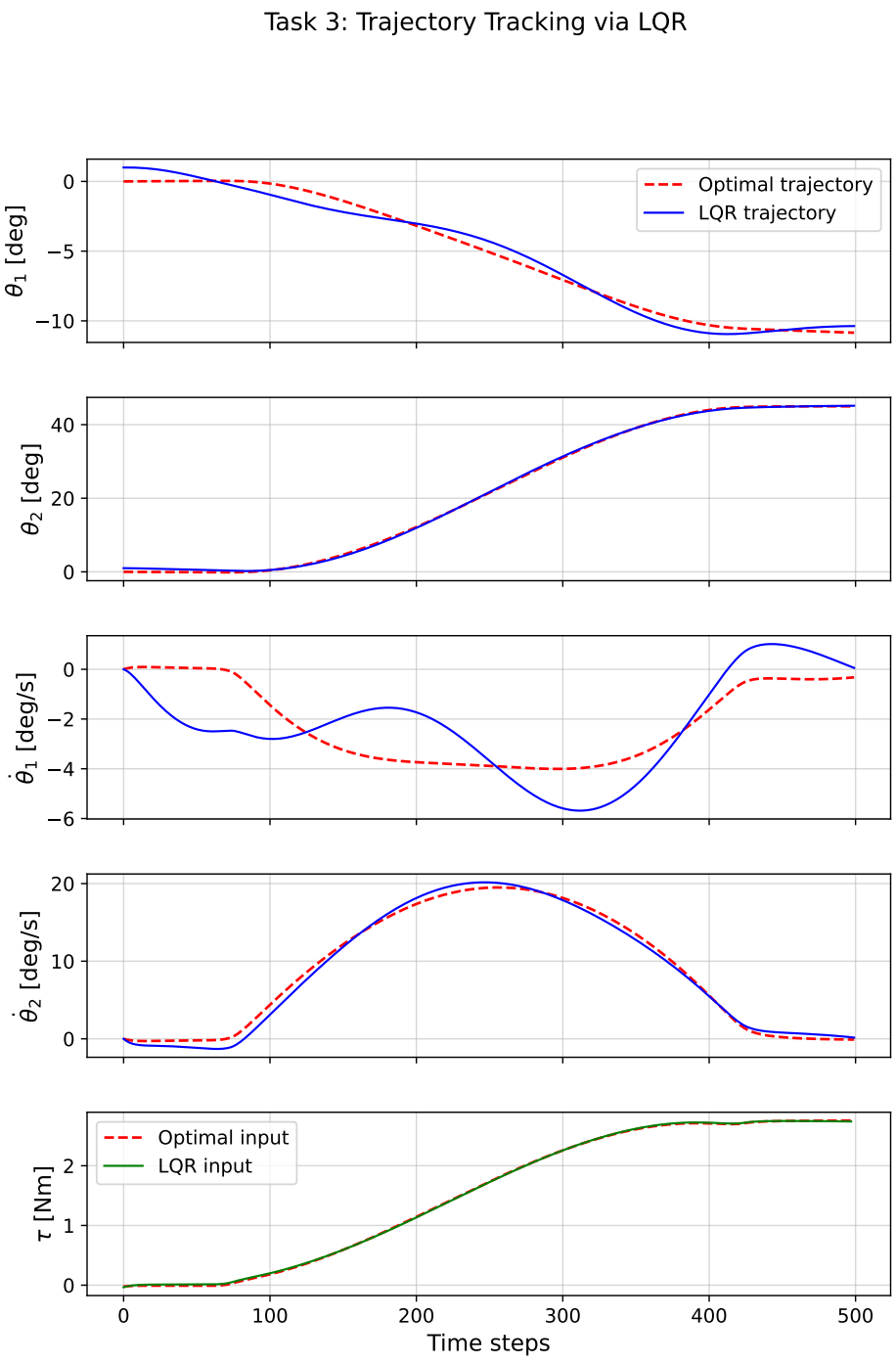


Figure 4.1: LQR Trajectory Tracking

Chapter 5

Task 4 - Trajectory tracking via MPC

In this task, we design a Model Predictive Control (MPC) scheme to track the optimal trajectory generated in Task 2. Unlike the previous regulator, the MPC explicitly handles input and state constraints, making it a more suitable choice for real-world applications where actuator saturation is a concern.

5.1 Introduction to MPC

MPC is a horizon control strategy that solves a constrained finite horizon optimal control problem at each sampling instant. Given the current state measurement x_t, the controller predicts the future evolution of the system over a prediction horizon T_{pred} and computes a sequence of optimal control inputs $u_{0,..,T-1}$ that minimizes a cost function while satisfying constraints. The key element is that only the first element of the optimal sequence, u_0^*, is applied to the plant and at the next time step and the optimization is repeated with the horizon shifted forward and the new state measurement x_t.

5.2 Implementation

The MPC solver was implemented using the `CasADi` framework with the `IPOPT` solver, ensuring robust handling of the optimization constraints.

To track the reference trajectory accurately, the nonlinear dynamics $\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$ are linearized at each step k of the prediction horizon around the reference point $(\mathbf{x}_k^{ref}, \mathbf{u}_k^{ref})$. Crucially, to account for the fact that $\mathbf{f}(\mathbf{x}^{ref}, \mathbf{u}^{ref}) \neq 0$ (since it is a trajectory, not an equilibrium), we include

an affine term $\mathbf{d}_k$ in the prediction model:

$$\mathbf{x}_{k+1} \approx A_k \mathbf{x}_k + B_k \mathbf{u}_k + \mathbf{d}_k \tag{5.1}$$

where $A_k = \nabla_{\mathbf{x}} \mathbf{f}|_{ref}$, $B_k = \nabla_{\mathbf{u}} \mathbf{f}|_{ref}$, and the affine correction term is computed as:

$$\mathbf{d}_k = \mathbf{x}_{k+1}^{ref} - (A_k \mathbf{x}_k^{ref} + B_k \mathbf{u}_k^{ref}) \tag{5.2}$$

This ensures that if the system is perfectly on the reference trajectory, the linearization error is zero.

Given state and input tracking error:

$$\begin{aligned} x_e &= x_k - x_k^{\text{ref}} \\ u_e &= u_k - u_k^{\text{ref}} \\ x_{eT} &= x_T - x_T^{\text{ref}} \end{aligned}$$

At each time step t it is solved the following constrained minimization problem, to minimize quadratic cost function.

$$\min_{\substack{x_{t+1},\ldots,x_{t+T} \\ u_t,\ldots,u_{t+T-1}}} \sum_{k=0}^{N-1} (x_e^\top Q x_e + u_e^\top R u_e) + x_{eT}^\top Q_T x_{eT}$$

$$\begin{aligned} \text{subj. to } & x_{k+1} = A_k x_k + B_k u_k + d_k, && \forall k \in [0, N-1] \\ & u_{min} \leq u_k \leq u_{max}, && \forall k \in [0, N] \\ & x_t = x_t^{\text{meas}} \end{aligned}$$

The horizon length is set to $N =$ `T_pred`. To handle the end of the simulation where the reference trajectory ends, the horizon is padded by replicating the final equilibrium state, ensuring the controller always has a valid target.

5.3 Results

The tracking performance of the MPC controller was tested under perturbed initial conditions ($x_0 = x_0^{ref} + \delta$) with $\delta = [1, 1]^\top$ degrees. The input saturation has a direct consequence on tracking accuracy. This results in a steady state error visibile in both θ_1 and θ_2, which fails to reach the final target angle.

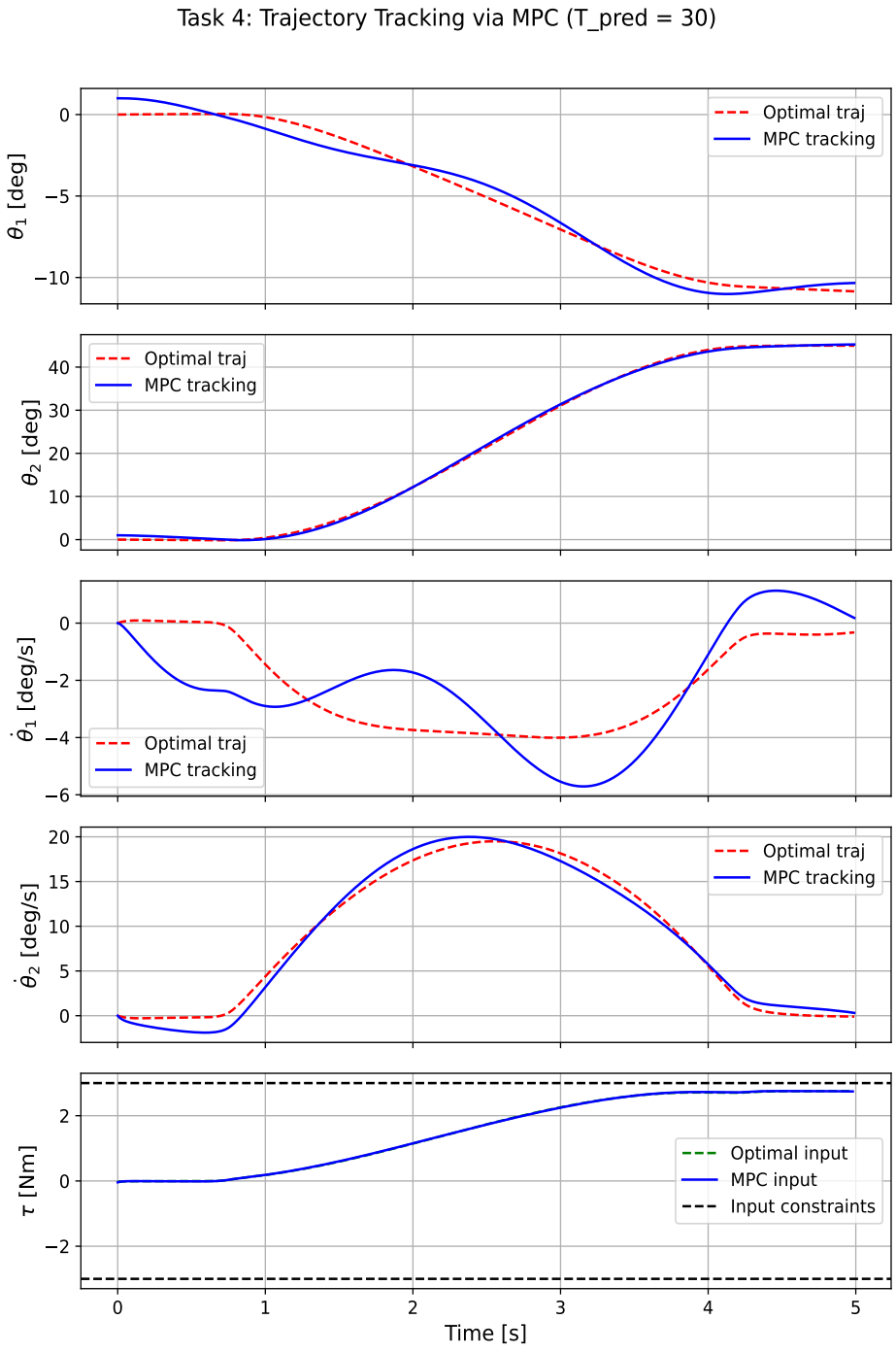


Figure 5.1: MPC Trajectory Tracking

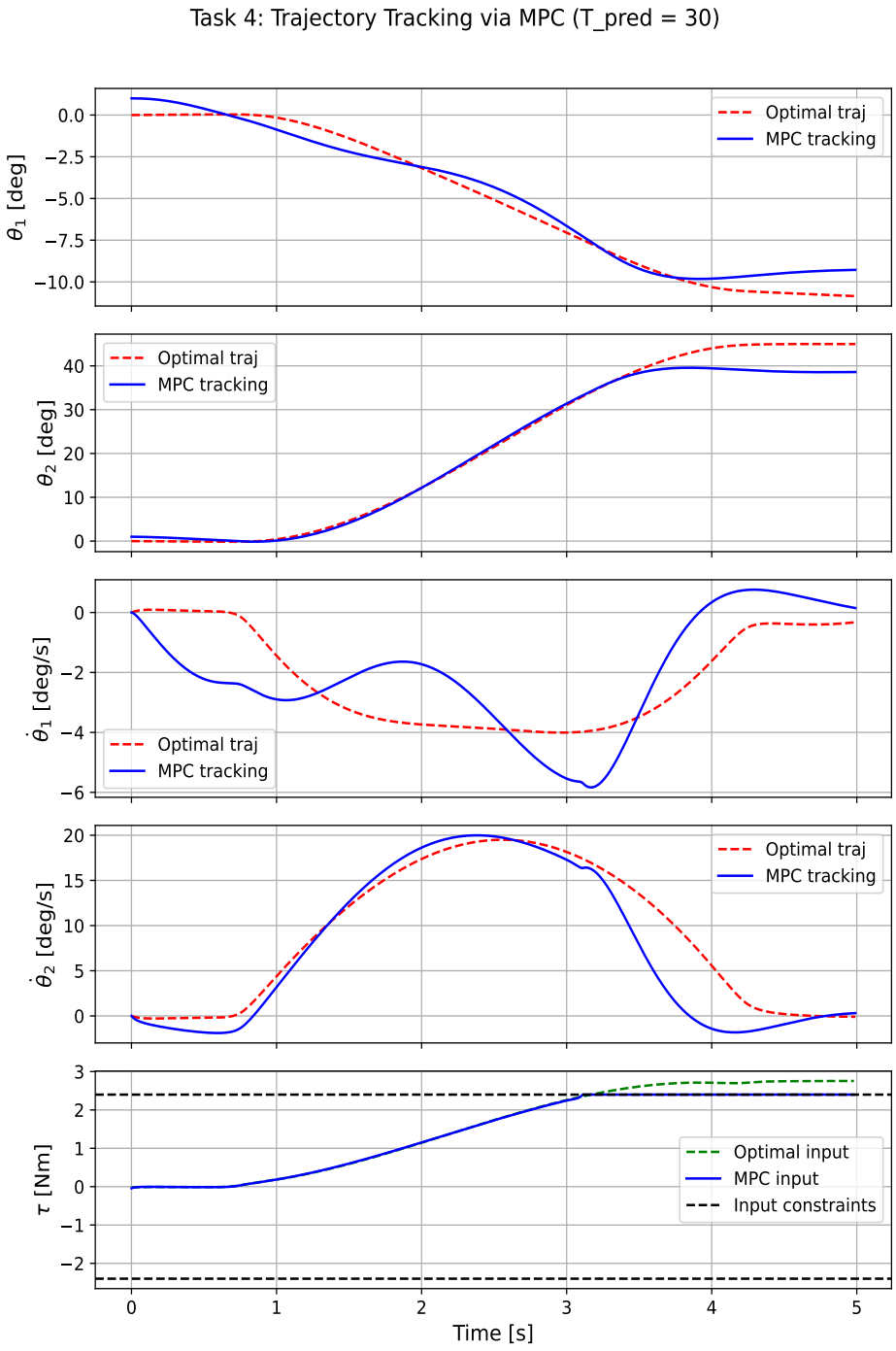


Figure 5.2: MPC Trajectory Tracking with Input Constraints

5.4 Better MPC constraint example

The previous example demonstrated the system's inability to reach the desired state due to insufficient motor torque. To better validate MPC robustness property, let us analyze a more critical scenario with the following initial conditions $x_0 = x_0^{ref} + \delta)$, $\delta = [5, 5]^\top$ and:

$$\theta_{1,\text{eq}} = 18.43, \quad \theta_{2,\text{eq}} = -90, \quad u_{\text{eq}} = -4.653$$
$$\theta_{1,\text{eq}} = -10.80, \quad \theta_{2,\text{eq}} = 45, \quad u_{\text{eq}} = 2.757$$

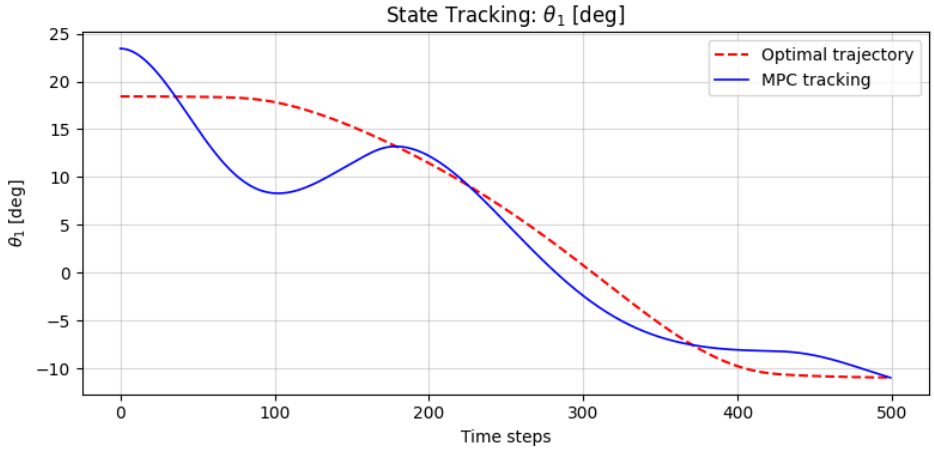


Figure 5.3: θ_1 evolution with MPC and limited input

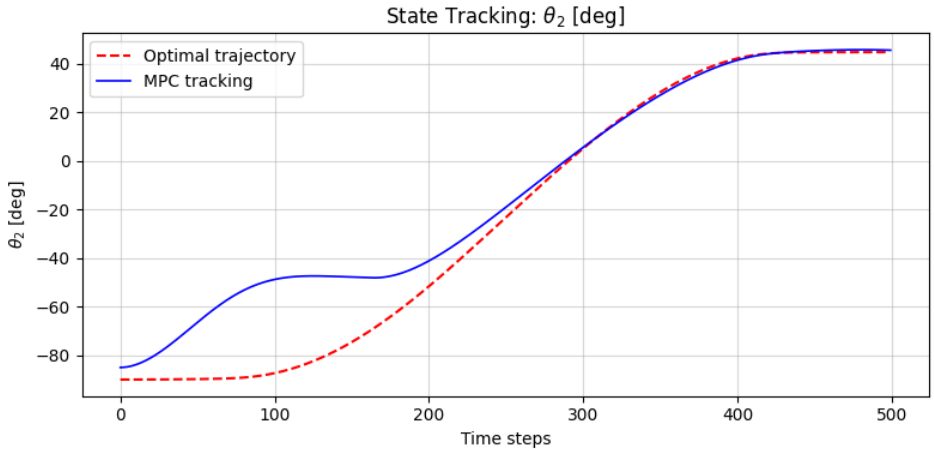


Figure 5.4: θ_2 evolution with MPC and limited input

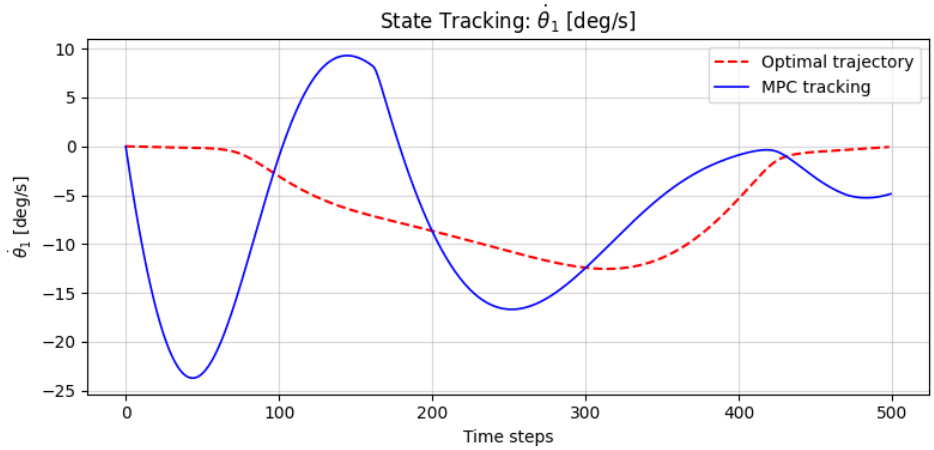


Figure 5.5: $\dot{\theta}_1$ evolution with MPC and limited input

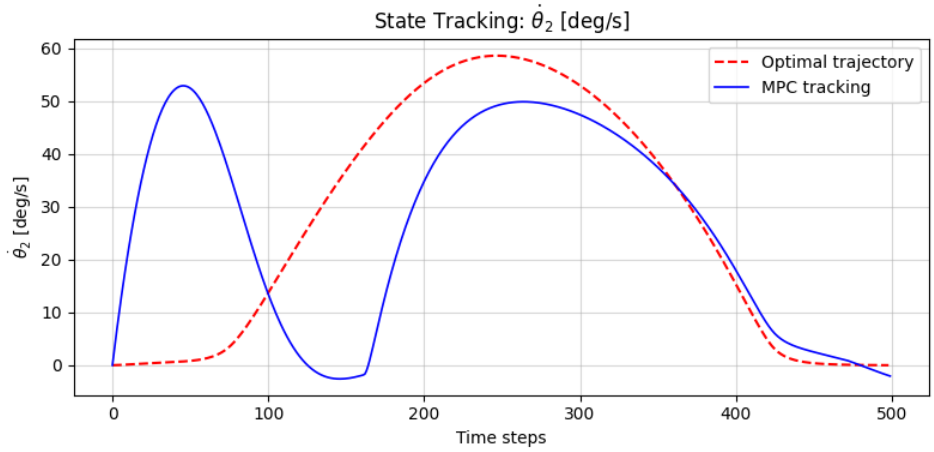


Figure 5.6: $\dot{\theta}_2$ evolution with MPC and limited input

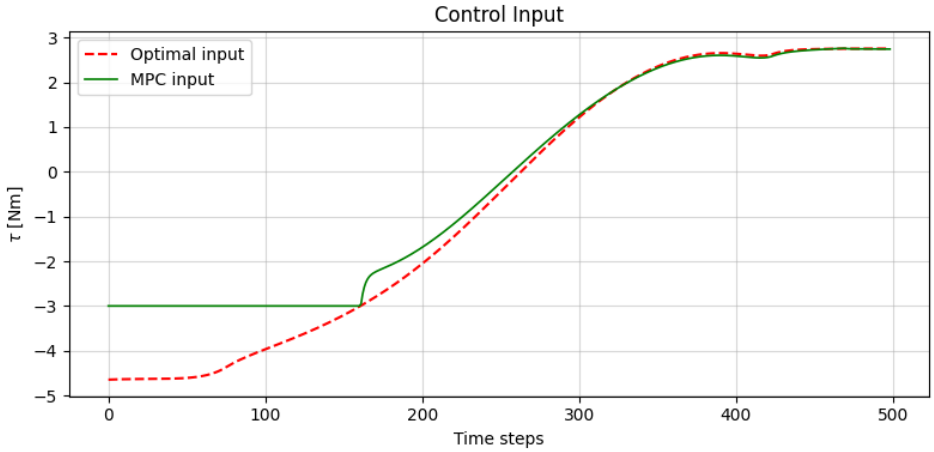


Figure 5.7: $\tau(t)$ evolution with MPC and limited input

The results demonstrate the robustness of the MPC scheme in the presence of initial state perturbations and actuator torque limitations. Although these constraints initially limit the controller's ability to accurately track the reference trajectory, **closed-loop stability is preserved**. As the required control effort decreases and the input constraints become inactive, the controller progressively recovers tracking performance.
This behavior confirms the effectiveness of MPC in handling disturbances and constraints while ensuring reliable long-term convergence toward the desired trajectory.

Conclusions

This work addresses the optimal control of a planar underactuated two link robotic manipulator. The system dynamics were discretized and the system's evolution was observed to be consistent and promising.

In Task 1, although significant results were achieved, the Armijo plots were sometimes not representative in the last iterations of the algorithm. Several potential causes could explain this issue. One possibility is numerical instability, which can occur when handling extremely large or small values and the precision is crucial. The use of a step reference may exacerbate this problem due to its lack of continuity.

In Task 2, a smooth reference curve was employed and the Armijo plots appeared much more accurate, with the descent direction consistently tangent to the cost function. Furthermore, the plot's scale indicated that the cost function's slope decreased over time, resulting in the Armijo method taking more steps as expected when nearing convergence.

In task 3, the Linear Quadratic Regulator control was successfully implemented to design an optimal feedback control. The results demonstrated that the controller could handle the initial perturbation.

In task 4, trajectory tracking using Model Predictive Control demonstrated robustness to both state and input perturbations. Even in the presence of an input constraint, the controller demonstrates good stability and real-time performances, progressively recovering from lost tracking.

The animations provided demonstrated satisfactory results across all tasks, as the references were closely followed and oscillations were eliminated through actuation.

Bibliography

[1] CasADi Developers. CasADi documentation. `https://web.casadi.org/docs/`, 2024. Accessed: 2024-05-21.

[2] CVXPY Developers. CVXPY documentation. `https://www.cvxpy.org/tutorial/`, 2024. Accessed: 2024-05-21.

[3] Giuseppe Notarstefano. Slides of the course, 2025. Prof. Giuseppe Notarstefano's slides of the course.

[4] SymPy Development Team. Sympy documentation. `https://docs.sympy.org/latest/index.html`, 2024. Accessed: 2024-05-21.

[3] [4] [2] [1]